CURSO · DÍA 2 DE 3

Dockerfile y aplicaciones .NET

Construir imágenes propias y contenerizar una API ASP.NET Core



Jorge López Blasco · Formador IT

Repaso rápido del día 1

1 ¿Imagen o contenedor?

¿Cuál es la plantilla de solo lectura y cuál la instancia en ejecución? ¿Cuántos contenedores pueden salir de una imagen?

2 ¿Qué significa -p 8080:80?

¿Cuál de los dos números es el puerto de vuestro equipo y cuál el del contenedor?

3 ¿Por qué se perdió el cambio?

En el lab 02 modificamos el index.html dentro del contenedor. ¿Por qué desapareció al hacer rm y volver a crearlo?

4 ¿Cómo se lleva una imagen sin internet?

Dos formas: una con ficheros y otra con un servicio dentro de vuestra red.

Agenda de hoy

09:00	Repaso del día 1
09:15	El Dockerfile: instrucciones, contexto y caché
09:55	**Lab 04 · Mi primer Dockerfile**
10:35	Imágenes de .NET y construcción multi-etapa
11:05	Descanso
11:25	**Lab 05 · Contenerizar una API .NET**
12:25	Configuración: una imagen, muchos entornos
12:45	**Lab 06 · Configuración**
13:25	.NET Framework y contenedores Windows
13:50	Resumen del día y preguntas

01

El Dockerfile

La receta reproducible de una imagen



De la imagen hecha a mano a la receta

docker commit

- Se modifica un contenedor a mano y se guarda
- No queda registro de qué se hizo
- No se puede revisar ni versionar
- Imposible de reproducir con exactitud
- El equivalente a un servidor configurado a mano

Dockerfile

- Un fichero de texto con los pasos, en orden
- Vive en Git, junto al código de la aplicación
- Se revisa en una pull request como cualquier cambio
- Cualquiera reconstruye la misma imagen con un comando
- Es la documentación exacta de cómo se construye

Las instrucciones que vais a usar

Instrucción	Qué hace
`FROM imagen AS nombre`	Imagen base de la que partimos. Toda imagen empieza por aquí
`WORKDIR /app`	Carpeta de trabajo dentro de la imagen (la crea si no existe)
`COPY origen destino`	Copia ficheros del contexto de construcción a la imagen
`RUN comando`	Ejecuta un comando durante la construcción: compilar, instalar paquetes…
`ENV CLAVE=valor`	Variable de entorno con un valor por defecto
`EXPOSE 8080`	Documenta en qué puerto escucha la aplicación
`USER usuario`	Usuario con el que se ejecutará el proceso
`ENTRYPOINT ["dotnet", "App.dll"]`	Proceso principal que se ejecuta al arrancar el contenedor
`CMD [...]`	Argumentos por defecto, que se pueden sustituir al ejecutar
`LABEL clave="valor"`	Metadatos descriptivos de la imagen

Un primer Dockerfile

labs/lab04-primer-dockerfile/Dockerfile

```
FROM nginx:alpine

LABEL org.opencontainers.image.title="mi-web"

COPY web/ /usr/share/nginx/html/

EXPOSE 80
```

FROM parte de la imagen oficial de nginx sobre Alpine.

LABEL añade metadatos, visibles con docker inspect.

COPY mete nuestra web en la carpeta que sirve nginx.

EXPOSE documenta el puerto.

Sin CMD: se hereda el arranque de nginx de la imagen base.

- `docker build -t mi-web:v1 .`

Qué ocurre en un docker build

docker build -t mi-web:v1 . — el punto es el contexto de construcción

1 · Contexto

El cliente empaqueta la carpeta indicada (el punto) y la envía al motor, excepto lo que excluya .dockerignore.

›

2 · Instrucciones

El motor ejecuta el Dockerfile línea a línea. Cada instrucción produce una capa.

›

3 · Caché

Si una instrucción y sus ficheros no han cambiado desde la última vez, reutiliza la capa: CACHED.

›

4 · Imagen

Se apilan las capas, se etiqueta con -t y queda lista para docker run o docker push.

.dockerignore funciona como .gitignore: deja fuera bin/, obj/, .git, .vs y cualquier secreto.

La caché de capas: el orden importa

Mal orden: cambio una línea de código

`FROM sdk:10.0`	CACHED
`COPY . .`	rehace
`RUN dotnet restore`	rehace
`RUN dotnet publish`	rehace

Como COPY . . incluye el código, cualquier cambio invalida el restore: se vuelven a descargar todos los paquetes NuGet.

Buen orden: cambio una línea de código

`FROM sdk:10.0`	CACHED
`COPY *.csproj .`	CACHED
`RUN dotnet restore`	CACHED
`COPY . .`	rehace
`RUN dotnet publish`	rehace

Las dependencias se restauran antes de copiar el código: el restore se reutiliza mientras no cambie el .csproj.

Mi primer Dockerfile

LAB

04

≈ 40 minutos

Escribir un Dockerfile, construir y versionar una imagen propia.

- Dockerfile para una web estática con nginx
- docker build y el contexto de construcción
- Versiones v1 y v2, y volver atrás en segundos
- Ver la caché de capas en acción
- Crear un .dockerignore e inspeccionar la imagen

`labs/lab04-primer-dockerfile/README.md`

02

.NET en contenedores

Imágenes oficiales y construcción multi-etapa



Imágenes oficiales de .NET

S

sdk:10.0

Compilador, CLI de dotnet y NuGet. Para construir. Es la más grande y nunca debería llegar a producción.

A

aspnet:10.0

Runtime de .NET y de ASP.NET Core. Para ejecutar APIs y aplicaciones web. Es la base habitual de producción.

R

runtime:10.0

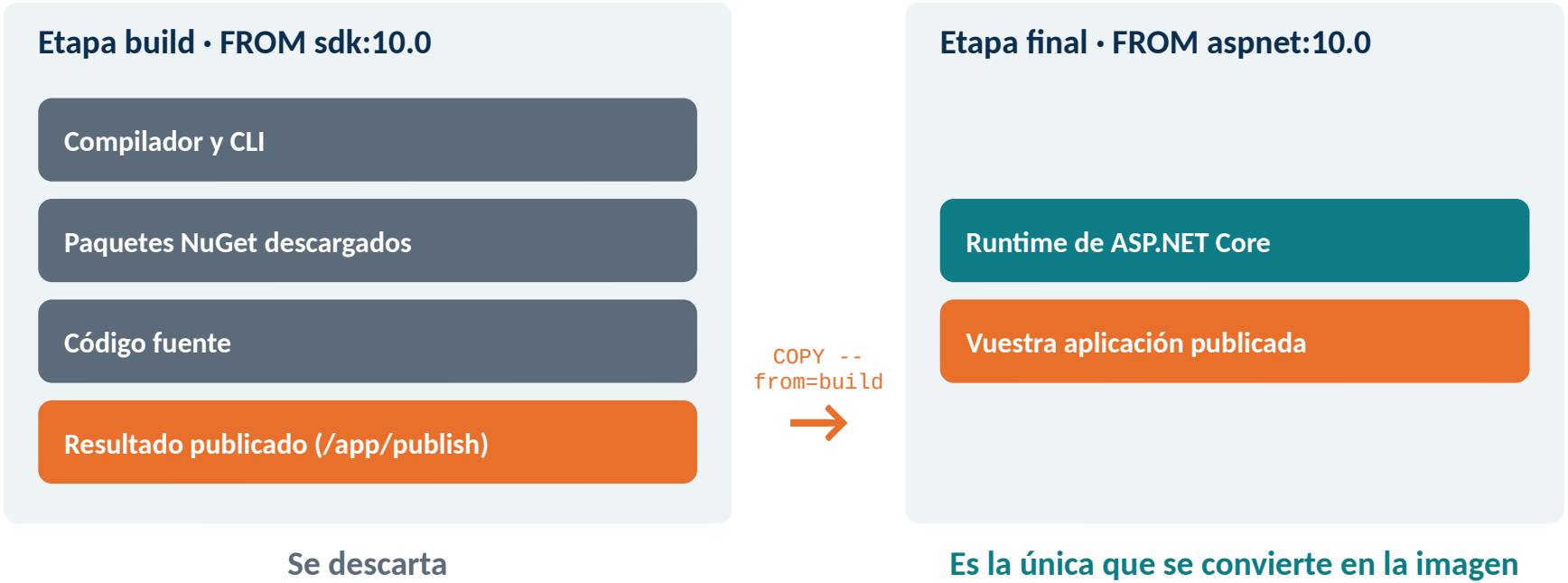
Solo el runtime de .NET, sin ASP.NET Core. Para aplicaciones de consola, servicios o procesos batch.

+

Variantes

-noble-chiseled: mínimas, sin shell ni gestor de paquetes. -alpine: más pequeñas. runtime-deps: para aplicaciones autocontenidas.

Construcción multi-etapa



El Dockerfile multi-etapa de una API .NET

ConvocatoriasApi/Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

COPY ConvocatoriasApi.csproj ./
RUN dotnet restore

COPY . .
RUN dotnet publish -c Release \
    -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .

EXPOSE 8080
USER $APP_UID

ENTRYPOINT ["dotnet", "ConvocatoriasApi.dll"]
```

Etapa build: SDK, restore primero (caché) y publicación.

Etapa final: solo el runtime y la carpeta publicada.

USER $APP_UID: se ejecuta con el usuario sin privilegios app.

8080: puerto por defecto desde .NET 8.

- **El código de la API no cambia.**

Tres detalles de las imágenes modernas de .NET

1 Puerto 8080

Desde .NET 8, ASP.NET Core escucha por defecto en el 8080 dentro del contenedor. Se publica en el puerto que queráis: -p 80:8080. Configurable con ASPNETCORE_HTTP_PORTS.

2 Usuario sin privilegios

Las imágenes incluyen el usuario app. Con USER $APP_UID la aplicación no se ejecuta como root, lo que limita mucho el daño si alguien la comprometiera.

3 Kestrel, no IIS

Dentro del contenedor la API la sirve Kestrel, el servidor web de ASP.NET Core. Delante se coloca un proxy inverso (nginx, IIS…) que veremos el día 3.

Contenerizar una API .NET

LAB

05

≈ 60 minutos

Empaquetar la API de convocatorias en .NET 10 como se hace en la práctica.

- Versión ingenua en una sola etapa y su tamaño
- Versión multi-etapa con .dockerignore
- Comparar tamaños y comprobar el usuario app
- La caché de restore en acción
- Tres instancias de la misma imagen

`labs/lab05-api-dotnet/README.md`

03

Configuración por entorno

Una imagen, muchos entornos



Cómo combina ASP.NET Core la configuración

Cada fuente sobrescribe a las anteriores. Las variables de entorno ganan a los ficheros.

appsettings.json

Valores por defecto, dentro de la imagen.

›

appsettings. {Entorno}.json

Según ASPNETCORE_ENVIRONMENT. Puede montarse desde fuera.

›

Variables de entorno

docker run -e … --env-file environment: en Compose

›

Línea de comandos

Argumentos al arrancar. Poco habitual en contenedores.

ConnectionStrings:Default → ConnectionStrings__Default (el doble guion bajo sustituye a los dos puntos)

Construir una vez, desplegar en cualquier entorno

Hoy: un paquete por entorno

- Transformaciones de web.config por entorno
- Se compila o empaqueta para cada destino
- Lo probado en preproducción no es exactamente lo que llega a producción
- Las diferencias aparecen en el peor momento



Con contenedores: una imagen

- Una sola imagen, construida una vez
- La configuración se inyecta al arrancar: variables o ficheros montados
- Preproducción y producción ejecutan el mismo binario
- Volver atrás es cambiar la etiqueta de la imagen

Configuración

LAB

06

≈ 40 minutos

Configurar la API desde fuera sin reconstruir la imagen.

- Sobrescribir appsettings con variables de entorno
- Ficheros de variables con --env-file
- Montar un appsettings.Production.json desde fuera
- El mismo artefacto en tres entornos
- Cambiar el puerto y diagnosticar con docker logs

`labs/lab06-configuracion/README.md`

04

.NET Framework y contenedores Windows

Qué implica y cuándo merece la pena



Contenedores Windows: lo que hay que saber

1 Host Windows obligatorio

.NET Framework solo corre en Windows, así que necesita contenedores Windows: Docker Desktop en modo Windows o Windows Server 2019/2022/2025 con runtime de contenedores.

2 Versión acoplada

La versión de Windows de la imagen (ltsc2019, ltsc2022, ltsc2025) debe ser compatible con la del servidor. Actualizar el host puede obligar a reconstruir imágenes.

3 Imágenes de varios GB

La base windowsservercore ocupa varios GB, frente a decenas o pocos cientos de MB de una imagen Linux. Más almacenamiento y despliegues más lentos.

4 Dependencias a reproducir

Componentes COM, fuentes, certificados, configuración de IIS, autenticación Windows (gMSA)… todo lo instalado a mano hay que llevarlo al Dockerfile.

Así sería una aplicación ASP.NET clásica

referencia/dotnet-framework/Dockerfile

```
# escape=`
FROM mcr.microsoft.com/dotnet/framework/sdk:4.8.1-windowsservercore-
ltsc2022 AS build
WORKDIR C:\src
COPY *.sln .
COPY MiAplicacionWeb\*.csproj MiAplicacionWeb\
COPY MiAplicacionWeb\packages.config MiAplicacionWeb\
RUN nuget restore
COPY MiAplicacionWeb\. MiAplicacionWeb\
WORKDIR C:\src\MiAplicacionWeb
RUN msbuild /p:Configuration=Release `
    /p:DeployOnBuild=true /p:WebPublishMethod=FileSystem `
    /p:PublishUrl=C:\publish

FROM mcr.microsoft.com/dotnet/framework/aspnet:4.8.1-
windowsservercore-ltsc2022
WORKDIR C:\inetpub\wwwroot
COPY --from=build C:\publish\. .
```

Mismas ideas: multi-etapa, restore primero, solo lo publicado al final.

La imagen aspnet ya trae IIS y ASP.NET 4.8.

Etiqueta ltsc2022: debe casar con el servidor.

Plantilla completa en referencia/dotnet-framework.

Estrategia recomendada

Contenedores Windows (Framework)

- Posible, y útil en casos concretos: retirar servidores antiguos, estandarizar despliegues de una aplicación que no se va a migrar pronto
- Coste alto: imágenes grandes, hosts Windows, versión acoplada, dependencias a reproducir
- Retorno limitado frente al esfuerzo

Contenerizar al migrar a .NET

- Cada aplicación que migréis a .NET 10 se contenedoriza con imágenes Linux pequeñas
- Hosts Linux más ligeros y baratos, ecosistema completo
- Migración y contenerización se hacen en el mismo esfuerzo y se refuerzan
- Los contenedores Windows, solo donde de verdad se justifiquen

Resumen del día

1 Dockerfile

La receta reproducible de una imagen. Vive en Git. Ordenad las instrucciones de lo que menos cambia a lo que más.

2 Multi-etapa

Se compila con sdk y se ejecuta con aspnet. A producción solo llega lo publicado, con el usuario app y en el puerto 8080.

3 Configuración

Una imagen para todos los entornos. Variables de entorno (doble guion bajo) y ficheros montados en lugar de transformaciones.

4 .NET Framework

Posible con contenedores Windows, pero con coste. La estrategia: contenerizar a medida que se migra a .NET.

MAÑANA · DÍA 3

Datos, redes, Compose e infraestructura

La aplicación completa con su base de datos, buenas prácticas y qué hace falta para llevarlo a vuestros servidores. ¿Preguntas?

